

A Project System Design Document
on
**BundleWise: A Retail Analytics Tool for Smart
Bundling and Profit-Driven Pricing**



Submitted by

2448005: Abishai Tom

2448033: Liya Khatija

2448036: Mansi Sapariya

Project Guide: Dr. Apash Roy

For the course

Course Code: MDS481

Course Name: PROJECT-I

1. Introduction

1.1 Purpose

This design document is a technical blueprint for developing **BundleWise**, a web-based retail analytics system focused on **wise product bundling and dynamic pricing optimization**. The primary goal of this document is to outline the architectural structure, module designs, workflows, and technologies required to implement the system. It ensures that all stakeholders—including developers, testers, and domain experts—understand how the system will function, what components are involved, and how they interact.

The document provides in-depth information about:

- System modules and their roles (e.g., data ingestion, pattern mining, forecasting, and pricing)
- User interface and interaction flows
Data and workflow architecture
- Design assumptions, constraints, and goals
- Testing strategies and validation plans

This design is aligned with the previously defined Software Requirements Specification (SRS) and guides the system development lifecycle from coding to deployment and maintenance.

1.2 Intended Audience

The intended audience for this design document includes:

- **Software Developers** – To implement the system as per the design.
- **Test Engineers** – To verify each component's behavior and ensure system reliability.
- **System Architects** – To evaluate architectural decisions and ensure scalability and modularity.
- **Retail Domain Experts** – To validate business logic (bundling and pricing relevance).
- **Project Managers** – To track progress and ensure delivery matches design goals.
- **Academic Reviewers/Evaluators** – For research validation, evaluation, and documentation review.

1.3 Scope

BundleWise is a web-based analytics tool tailored for **small and medium-sized retailers**. It helps retailers analyze their **sales data** to identify frequently purchased product combinations, predict future sales trends, and generate intelligent pricing recommendations. The system supports decision-making through an interactive dashboard that presents:

- **Frequently bought product bundles** (even across separate transactions)
- **Sales and profit forecasts** using time series models
- **Dynamic pricing suggestions** to maximize revenue or clear slow inventory
- **Data visualizations and exportable reports** for business usage

It is designed to work even with **non-invoice-level data** (i.e., scattered transaction records), offering a practical solution for businesses without advanced technical infrastructure.

1.4 Definitions, Acronyms, and Abbreviations

- **BundleWise**: The proposed smart retail analytics system provides product bundling, forecasting, and dynamic pricing recommendations based on transaction data.
- **MBA (Market Basket Analysis)**: A data mining technique used to identify products frequently purchased together in a single transaction. It helps retailers understand item affinity and plan product bundles.
- **SPM (Sequential Pattern Mining)**: A technique for finding sequential purchasing patterns from transactional data spread across different time points or visits by the same customer.
- **SKU (Stock Keeping Unit)**: A unique identifier for each product or item a retailer sells, used for inventory tracking and sales analysis.
- **CSV (Comma-Separated Values)**: A simple text-based file format for storing tabular data, where commas separate values.
- **UI (User Interface)**: The visual layer of the application that users interact with, including buttons, forms, and dashboards.
- **POS (Point of Sale)**: A physical or digital system where retail transactions are completed, typically where the payment is made.
- **ARIMA (AutoRegressive Integrated Moving Average)**: A statistical model used in time series forecasting combines autoregressive, differencing, and moving average techniques.
- **LSTM (Long Short-Term Memory)**: A type of deep learning recurrent neural network (RNN) particularly well-suited for sequential data and long-range forecasting.

- **Prophet:** An open-source forecasting tool developed by Facebook (Meta), designed to produce quick and reliable time series forecasts with minimal configuration. Suitable for seasonal and irregular trends.
- **Pandas:** A Python library used for data manipulation and analysis, particularly for working with structured data in tabular form.
- **NumPy:** A foundational Python library used for numerical computing and handling arrays.
- **mlxtend:** A Python library that extends machine learning tools, commonly used for implementing frequent pattern mining algorithms like Apriori and FPGrowth.
- **SciPy:** A Python library for scientific and technical computing, including optimization and mathematical modeling.
- **Keras / TensorFlow:** Open-source libraries used to build and train deep learning models, including LSTM networks.
- **Streamlit / Dash:** Python-based web application frameworks used to create interactive and data-driven dashboards.

2. System Overview

2.1 Project Summary

BundleWise is an innovative retail analytics system that helps small and medium-sized retailers **identify frequently bought product bundles, forecast sales trends, and generate data-driven pricing strategies**. The system analyzes past sales transactions (uploaded as CSV files), discovers **sequential purchase patterns**, applies **time series forecasting**, and offers **dynamic pricing recommendations** that align with seasonal demands and business goals. BundleWise transforms complex analytics into a **simple, interactive dashboard**, allowing non-technical users to make informed decisions about product bundling, inventory planning, and pricing without requiring deep data science or machine learning knowledge.

2.2 Key Objectives and Features

BundleWise is designed with the following core objectives and functional features:

- **Data Upload and Preprocessing**
Retailers can upload historical sales data (CSV format), which the system validates, cleans, and structures for analysis.
- **Market Basket Analysis (MBA)**
Discover products frequently purchased together in a single transaction using algorithms like Apriori or FPGrowth. This forms the foundation for identifying popular product combinations and designing bundles.
- **Smart Product Bundling using Sequential Pattern Mining (SPM)**
Discover products not just bought together in a single transaction, but purchased sequentially across time (e.g., shampoo → conditioner → hair mask).
- **Sales and Profit Forecasting**
Predict upcoming product demand and profitability trends using time series models like ARIMA, Prophet, and LSTM.
- **Dynamic Pricing Optimization**
Suggest price adjustments for products and bundles to maximize profit, clear slow-moving stock, or respond to seasonal demand changes.
- **Interactive Dashboard and Visualization**
View insights through charts, graphs, and tables using an intuitive user interface—filter, explore, and interpret data easily.
- **Reporting and Export Functionality**
Export forecast results, bundling reports, and pricing strategies in formats such as PDF and Excel for offline use.

2.3 System Environment

The BundleWise system will operate in the following technical environment:

- **Web-Based Interface**
Fully accessible through standard web browsers—no installation required.
- **CSV Data Interaction**
Users upload sales data files in CSV format; the system ingests, validates, and processes them internally.
- **Cloud Hosting**
For scalable and reliable access, the application will be deployed using **Streamlit Cloud**, **Render**, or a Flask-based server.
- **Technologies and Tools Used:**
 - **Programming Language:** Python
 - **Data Handling:** Pandas, NumPy
 - **Pattern Mining:** PrefixSpan / SPADE (via mlxtend or SPMF)
 - **Forecasting:** ARIMA, Facebook Prophet, LSTM (via TensorFlow/Keras)
 - **Optimization:** SciPy (for linear programming and optimization)
 - **UI and Visualization:** Streamlit, Dash, Plotly
 - **Report Export:** FPDF / xlsxwriter for generating downloadable outputs

The environment is designed to support easy interaction, efficient computation, and portability across platforms.

3. Design Considerations

3.1 Assumptions

The following assumptions are made during the design and development of the BundleWise system:

- **Structured CSV Input:** It is assumed that users will upload sales data in a structured CSV format, containing essential fields like Product ID, Transaction Date, Quantity Sold, and Customer ID (if applicable).
- **Historical Data Availability:** Retailers will have sufficient historical data to enable pattern recognition and forecasting models to function effectively.
- **Users' Basic Digital Literacy:** Users will have basic experience with file uploading, navigating a web dashboard, and understanding simple visual reports.
- **No Real-Time Requirements:** Users do not expect real-time integration or instant analysis; minor processing time for results is acceptable.
- **Static or Uploaded Seasonal/Event Data:** The system assumes that the user will fix or manually provide any data about seasonal trends or local events (e.g., festival periods).

3.2 Constraints

Certain limitations may impact how the system operates or expands:

- **No POS or ERP Integration (Initial Release):** The system does not directly connect with Point-of-Sale (POS) or Enterprise Resource Planning (ERP) software. All input must be uploaded manually as CSV files.
- **Internet Dependency:** As a cloud-hosted web application, users must have a stable internet connection to access the tool and upload/download data.
- **Limited Hosting Resources:** If hosted on platforms like Streamlit Cloud (free tier), system performance may be affected by storage, memory, or processing limits.
- **Algorithmic Boundaries:** Machine learning models (e.g., LSTM, Prophet) may have performance or accuracy issues with incomplete, inconsistent, or small datasets.
- **Data Privacy Regulations:** The system must comply with data privacy standards. It does not store sensitive user data beyond temporary processing unless explicitly designed to do so.

3.3 Design Goals

The BundleWise system is built to meet the following design goals:

- **Usability:** The user interface should be intuitive and straightforward enough for non-technical retail users to upload data, view results, and export reports without technical help.
- **Modularity:** Each component (e.g., pattern mining, forecasting, pricing optimization) should function independently, making the system easy to update or expand.
- **Scalability:** The backend should be capable of handling increased data volume or computation as the business size or traffic grows.
- **Maintainability:** The codebase will be clean, well-commented, and modular to allow easy debugging, upgrading, or enhancement.
- **Interpretability:** Outputs, especially analytics and forecasts, should be visualized with charts and tables that help users understand and act on insights.

3.4 Technologies and Tools Used

Component	Technology/Tool	Purpose
Programming Language	Python	Core language for backend logic and analytics
Data Handling	Pandas, NumPy	Data ingestion, cleaning, transformation, and manipulation
Market Basket Analysis (MBA)	mlxtend.frequent_patterns (Apriori, FPGrowth)	Discovering frequently co-purchased items within single transactions
Sequential Pattern Mining (SPM)	SPMF, mlxtend (PrefixSpan, SPADE)	Identifying product sequences across different customer visits
Time Series Forecasting	Prophet, ARIMA (statsmodels), LSTM (Keras)	Predicting future product demand and profit trends

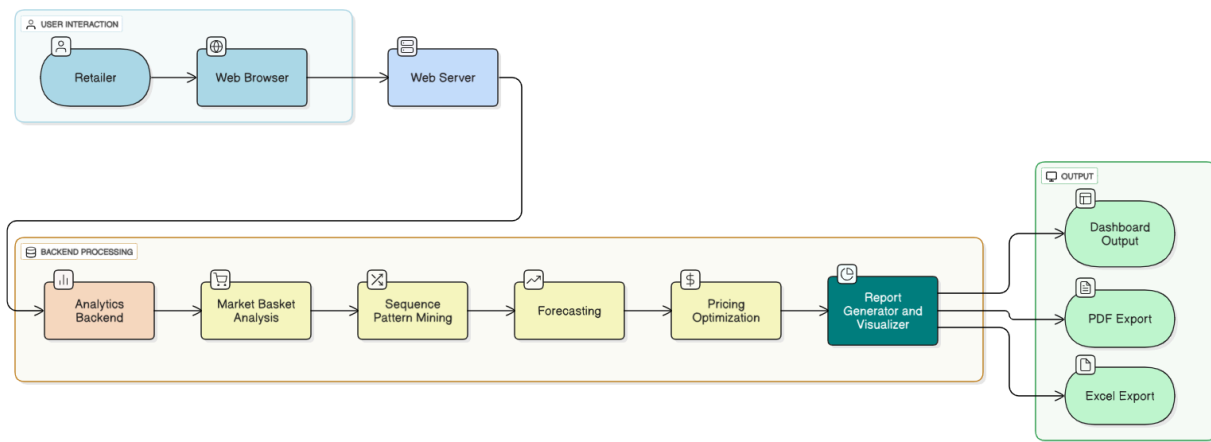
Optimization	SciPy (optimize), PuLP (optional)	Dynamic pricing optimization using linear programming
UI Development	Streamlit, Dash, Plotly	Building intuitive and interactive dashboards
Deployment	Flask, Streamlit Cloud, Render	Hosting the application for web access
Reporting	FPDF, xlsxwriter	Generating PDF/Excel reports from analysis results

4. System Architecture

4.1 High-Level Architecture Diagram

The BundleWise system follows a **client-server architecture**, where the **frontend (user interface)** communicates with the **backend server**, which performs data processing, analysis, and visualization.

High-Level Flow :



- **Users upload sales data (.csv)** through the web UI.
- **Backend processes the data** using market basket, pattern mining, forecasting, and optimization models.
- **The dashboard displays the insights**, and users can download reports.

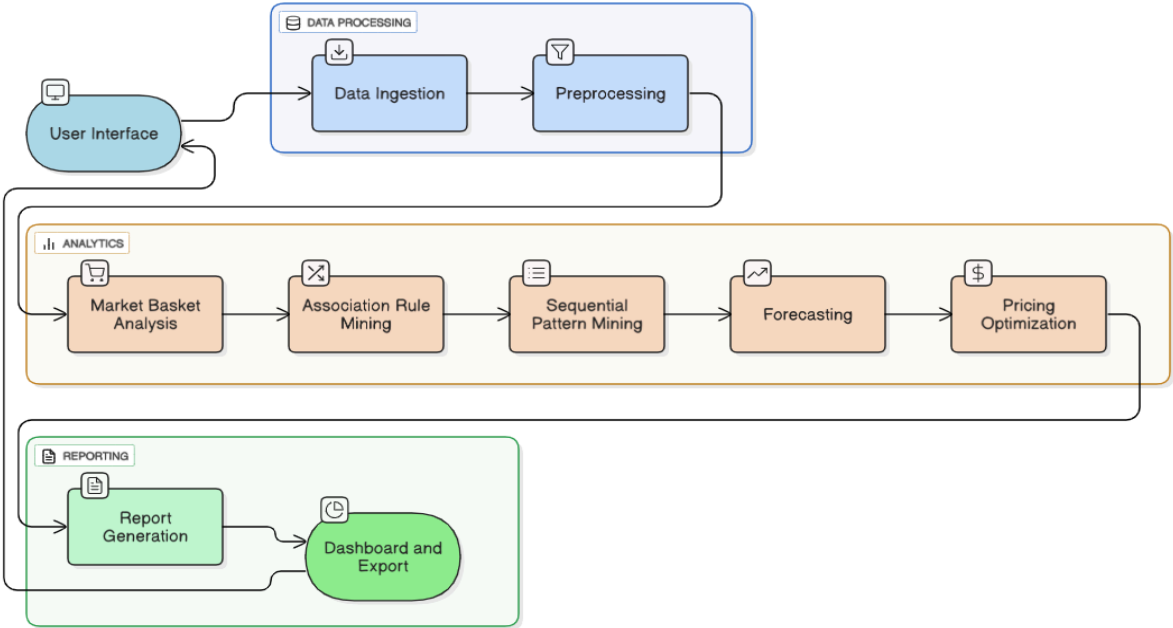
4.2 Component Diagram

Internal Modules and Their Relationships

This diagram describes the internal modules of the system and how they are related:

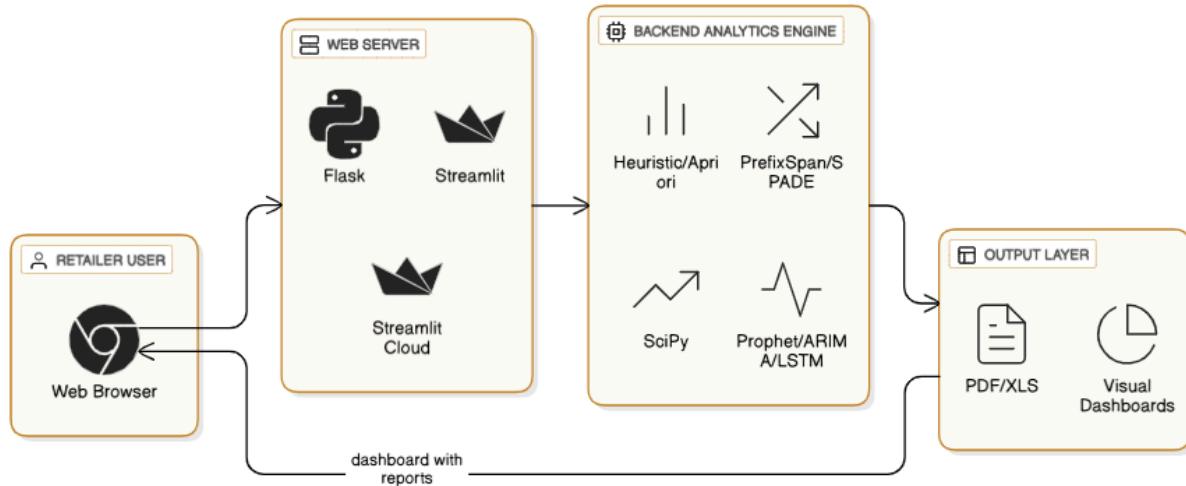
Module	Description
User Interface (UI)	Web-based interface using Streamlit or Dash. Accepts uploads, displays results.

Data Ingestion Module	Reads CSV files, validates format, and handles missing data.
Preprocessing Module	Cleans data, formats timestamps, and transforms input for model compatibility.
Market Basket Analysis Module (MBA)	Applies Apriori or FPGrowth algorithms to discover products frequently bought together in the transaction.
Sequential Pattern Mining Module (SPM)	Uses PrefixSpan or SPADE to extract sequential product purchase patterns across multiple visits.
Forecasting Module	Uses Prophet, ARIMA, or LSTM to predict sales and profit trends.
Pricing Optimization Module	Suggests optimal prices for products/bundles using SciPy-based optimization.
Reporting Module	Converts analytical output into downloadable PDF/Excel reports.



4.3 Deployment Diagram

Describes **where components are hosted** and how users interact with the deployed application.



4.4 Design Considerations

Here are the major **design principles and priorities** that influenced the architecture of BundleWise:

Modularity

- Each functional block (e.g., mining, forecasting) is a self-contained module.
- Makes the system easier to test, debug, and upgrade in the future (e.g., switch from Prophet to NeuralProphet).

Cloud Compatibility

- The system is designed to run seamlessly on cloud platforms like Streamlit Cloud or Render.
- This allows retailers to access the system from anywhere without needing installation.

Data Privacy

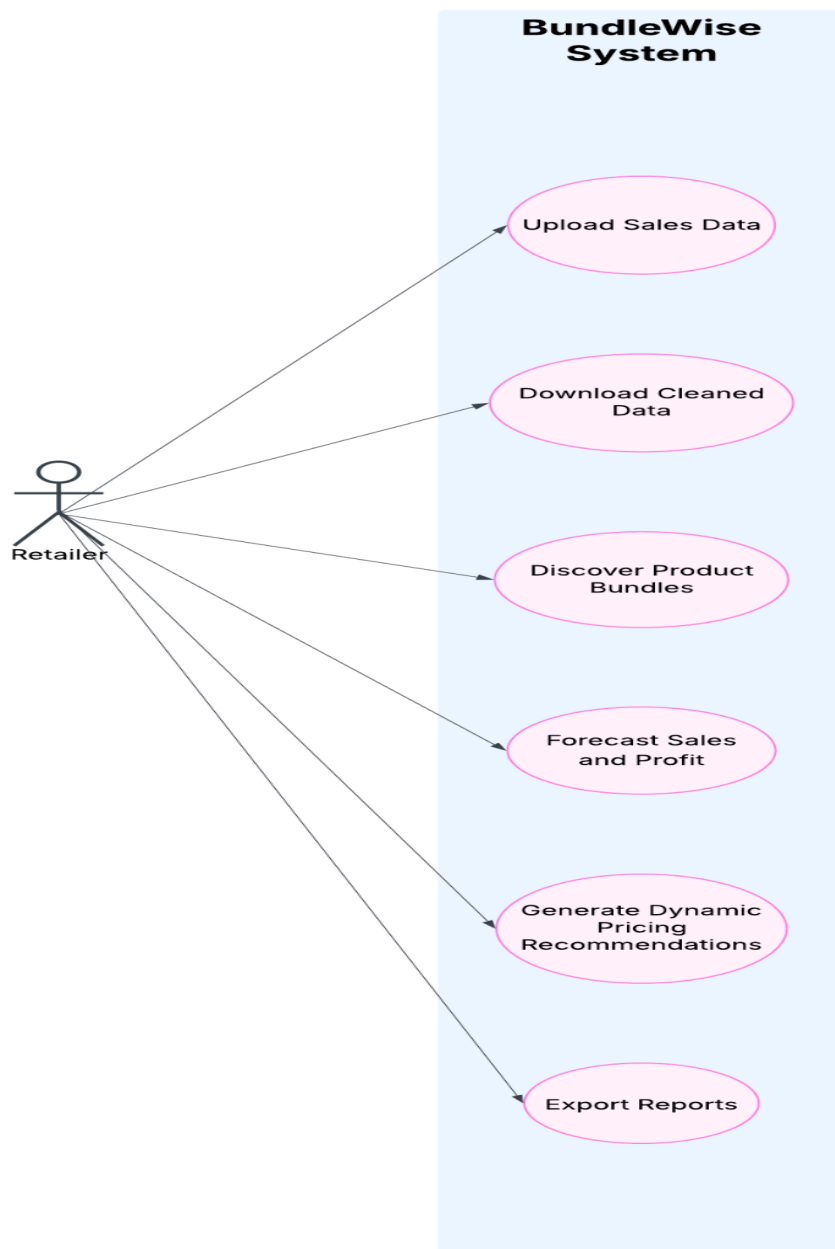
- User data (CSV files) is only stored temporarily during the session.
- No personal customer data is required, and basic security practices (e.g., file validation, HTTPS) are followed.

- The system complies with good privacy practices, which is especially useful if scaling to GDPR-compliant regions.

Low Dependency on Infrastructure

- No real-time system hooks (e.g., POS API), so it's lightweight and flexible.
- Does not require expensive hardware or backend systems to run efficiently.

4.5. Use Case Diagram



Use Case 1: Upload Sales Data

- **Actor:** Retailer
- **Description:** Retailer uploads historical sales data in CSV format through the dashboard.
- **Preconditions:** User has a structured CSV file with required fields (e.g., ProductID, CustomerID, Date, Quantity, Price).
- **Postconditions:** Data is validated, cleaned, and temporarily stored.
- **Basic Flow:**
 1. Retailer logs into BundleWise.
 2. Navigates to the "Upload" section.
 3. Upload the CSV file.
 4. The system validates the format and structure.
 5. Confirmation is shown to the user.

Use Case 2: Download Cleaned Data

- **Actor:** Retailer
- **Description:** Allows the user to download the preprocessed/cleaned version of the uploaded CSV file.
- **Preconditions:** Upload process is completed successfully, and the data is cleaned.
- **Postconditions:** The cleaned CSV file is downloaded.
- **Basic Flow:**
 1. After uploading, the system cleans and transforms the data.
 2. A "Download Cleaned Data" button appears.

3. The user clicks to download the cleaned file.

Use Case 3: Discover Product Bundles (MBA + SPM)

- **Actor:** Retailer
- **Description:** System identifies products frequently bought together using Market Basket Analysis (MBA) and Sequential Pattern Mining (SPM).
- **Preconditions:** Cleaned data is available.
- **Postconditions:** Frequent itemsets and sequences are displayed with support/confidence/lift.
- **Basic Flow:**
 1. User clicks "Analyze Bundles".
 2. MBA is applied (Apriori/FPGrowth).
 3. SPM is applied (PrefixSpan/SPADE).
 4. Results are shown with metrics.
 5. The user explores them visually.

Use Case 4: Forecast Sales and Profit

- **Actor:** Retailer
- **Description:** Forecasts future trends for sales and profit using models like Prophet, ARIMA, or LSTM.
- **Preconditions:** Historical time-series data is cleaned and grouped.
- **Postconditions:** Forecast graphs and tables are generated.
- **Basic Flow:**

1. The user selects a product or bundle and a time range.
2. The system applies forecasting models.
3. Forecast results are visualized and downloadable.

Use Case 5: Generate Dynamic Pricing Recommendations

- **Actor:** Retailer
- **Description:** Suggests optimized prices for bundles/products using forecasting and optimization techniques.
- **Preconditions:** Forecasted data and bundling insights are available.
- **Postconditions:** Suggested prices and expected sales uplift are shown.
- **Basic Flow:**
 1. The user opens the "Pricing" panel.
 2. The system calculates optimal prices.
 3. Pricing suggestions are shown.

Use Case 6: Export Reports

- **Actor:** Retailer
- **Description:** Allows users to download insights and results in PDF or Excel format.
- **Preconditions:** Analytics modules have generated output.
- **Postconditions:** Reports are downloaded.
- **Basic Flow:**
 1. The user clicks on the "Export" button.

2. Select the type of report.
3. The system generates and offers a PDF/Excel download.

5. Detailed Module Design

5.1 User Interface Module

Function:

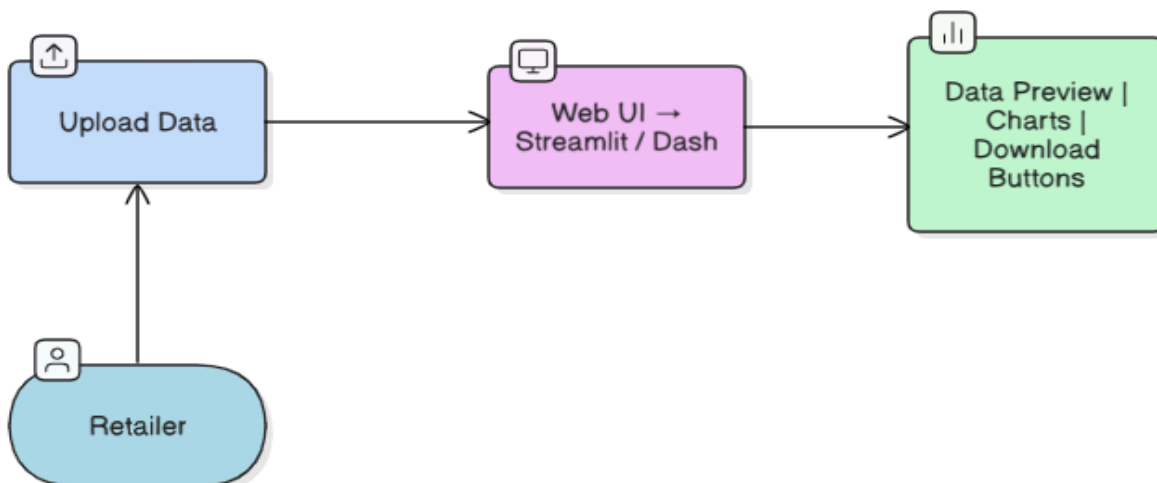
Provides an intuitive and interactive web interface through which users interact with the system.

Workflow:

- The user visits the web page hosted via Streamlit/Dash.
- The upload button allows selection of a .csv sales dataset.
- Once uploaded, the system displays progress/status.
Results are displayed in dashboard panels (e.g., bundles, forecasts, pricing) upon processing.
- Provides options to filter data, view tooltips, and export reports.

Tools Used:

- Streamlit / Dash (Plotly) for dashboard and UI rendering
- HTML/CSS for styling
Python callbacks for interactivity



5.2 Data Ingestion & Preprocessing Module

Function:

Responsible for reading raw sales data, validating format, cleaning inconsistencies, and transforming it into an analyzable format.

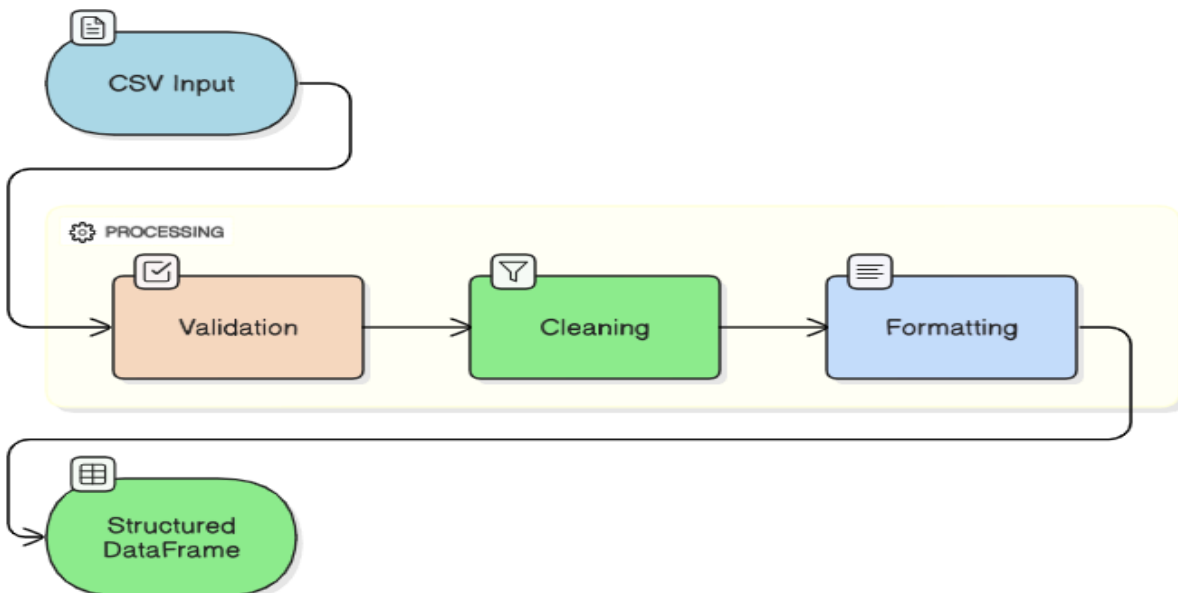
Workflow:

- Accept a .csv file with fields like Transaction ID, Customer ID, Product ID, Date, Quantity, and Price.
- Check for required fields and correct formatting.
- Handle missing/null values, duplicates, and invalid dates.
- Convert date columns into proper datetime objects.
- Group/aggregate records based on customer or transaction.

Tools Used:

- pandas for data handling
- datetime, re for parsing
- NumPy for transformation

Key Output: Cleaned DataFrame ready for mining or forecasting.



5.3 Market Basket Analysis (MBA) Module

Function:

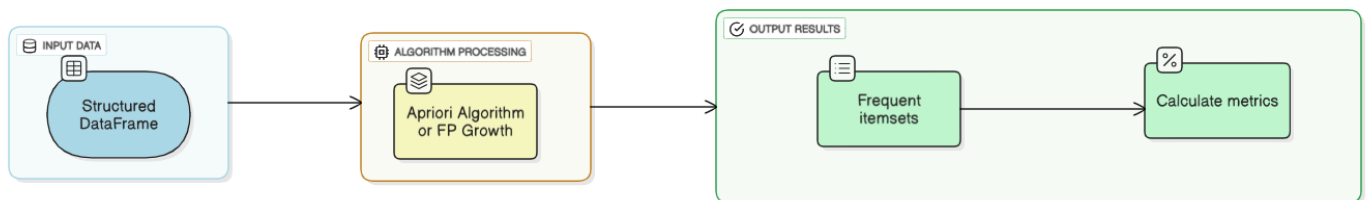
Identifies products frequently purchased together in the same transaction. This helps in understanding item affinity and supports practical bundling.

Workflow:

- Transform cleaned CSV data into transactional format (based on Invoice ID or Customer ID).
- Apply Apriori or FPGrowth algorithms.
- Calculate itemset support, confidence, and lift values.

Tools Used:

- `mlxtend.frequent_patterns` (Apriori, FPGrowth)
- Pandas for transaction formatting and encoding



5.4 Sequential Pattern Mining Module

Function:

Analyzes the structured transaction data to discover frequent product purchase sequences.

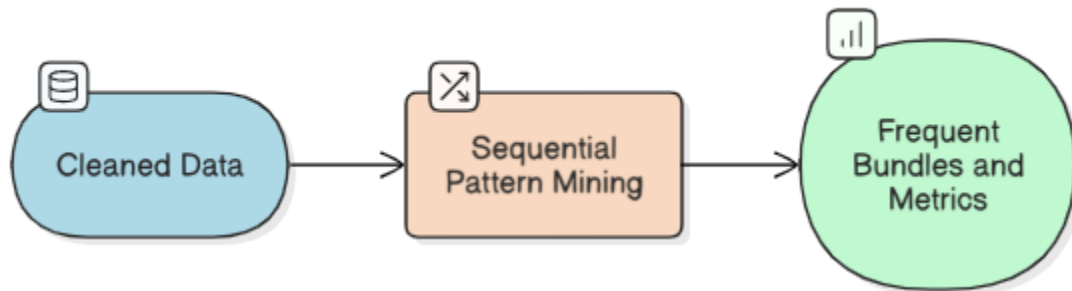
Workflow:

- Convert transaction data into sequences per user (e.g., visits over time).
- Apply **PrefixSpan** or **SPADE** algorithm.
- Extract frequent patterns (e.g., {Milk → Bread}, {Shampoo → Conditioner}).
- Calculate metrics: Support, Confidence.

Tools Used:

- `mlxtend`, `SPMF`, or a custom implementation of `PrefixSpan`

- collections or itertools for sequence generation



5.5 Time Series Forecasting Module

Function:

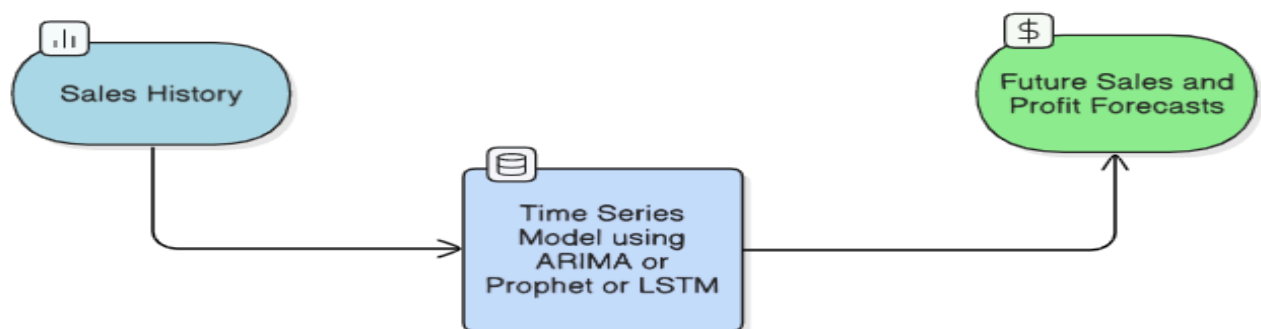
Predicts future product-level sales and profits using historical trends.

Workflow:

- Input: cleaned and grouped sales data (by week/month/product).
- Apply models like **Prophet**, **ARIMA**, or **LSTM**.
- Account for seasonality, promotions, and holidays if available.
- Output: Weekly/monthly forecast with confidence intervals.

Tools Used:

- Facebook Prophet – for interpretable and fast forecasting
- statsmodels – for ARIMA
- Keras / TensorFlow – for deep LSTM networks.



5.6 Pricing Optimization Module

Function:

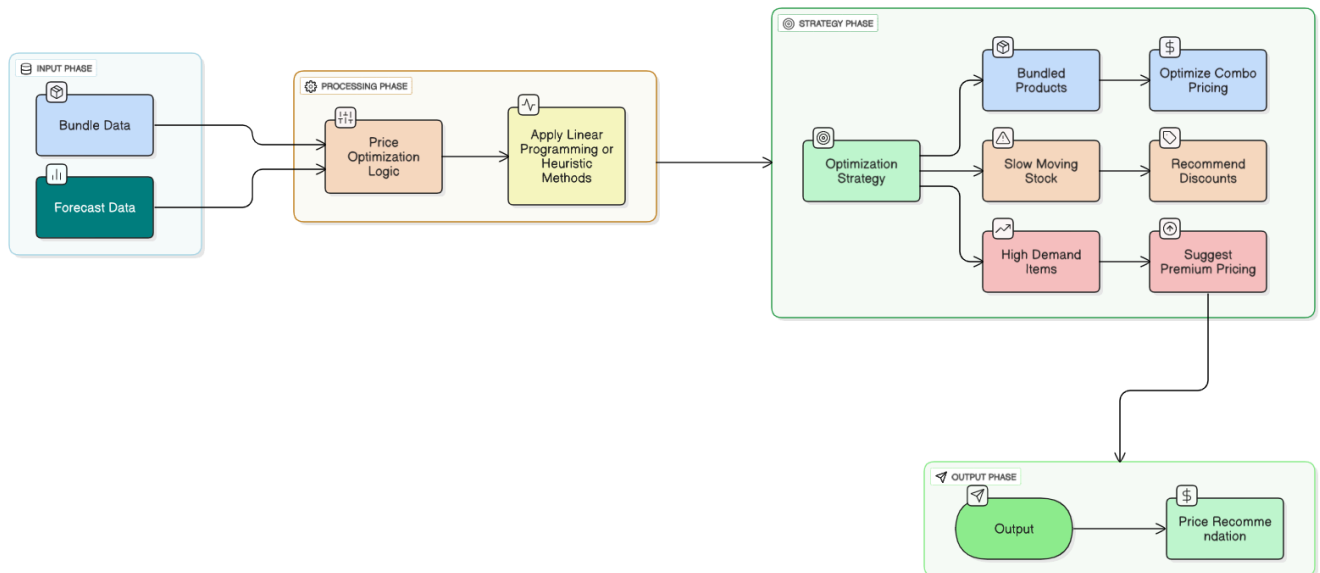
Uses mathematical optimization to suggest price adjustments for products or bundles, maximizing profit or sales.

Workflow:

- Inputs: Bundle data from MBA and SPM and forecast data.
- Apply **linear programming** or heuristic models.
- Optimize prices for:
 - Bundled products
 - Slow-moving stock (discounting)
 - High-demand items (premium pricing)
- Output: price recommendation for each product/bundle.

Tools Used:

- SciPy.optimize for Linear Programming
- Basic economic models (elasticity, break-even)
- pulp (optional for LP-based price modeling)



5.7 Reporting & Export Module

Function:

Compiles all analytics (bundles, forecasts, pricing) into visual and tabular reports exportable in formats like PDF or Excel.

Workflow:

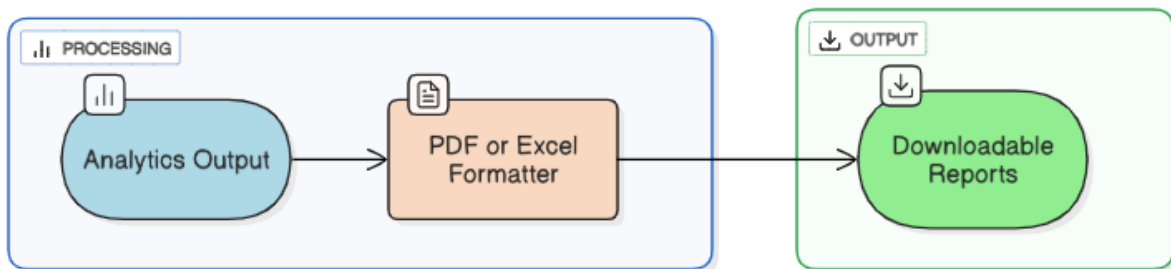
- Compile cleaned output data, visuals, and tables.
- Format for readability (add headers, charts, insights).
Export via Streamlit button as .xlsx or .pdf.

Tools Used:

- `pandas.to_excel`
- `plotly` or `matplotlib` for charts
- `pdfkit`, `fpdf`, or `ReportLab` for PDF export

Output:

- Bundle Report
- Forecast Report
- Pricing Strategy Summary

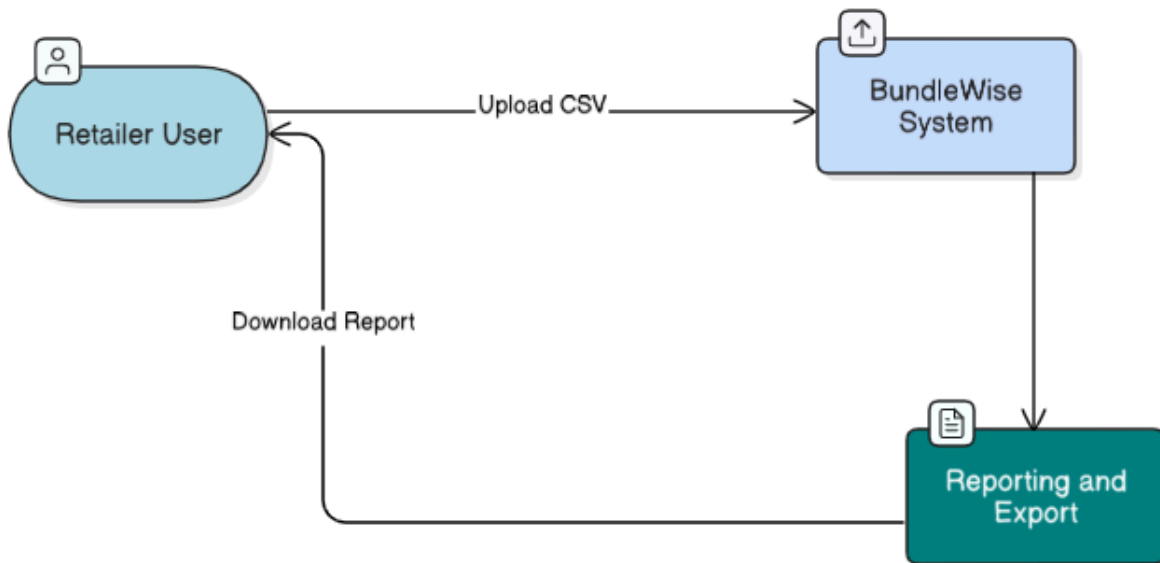


6. Data and Workflow Design

6.1 Data Flow Diagrams (DFD)

Level 0 DFD (Context-Level Diagram):

This diagram shows the system as a single process and how it interacts with external entities.

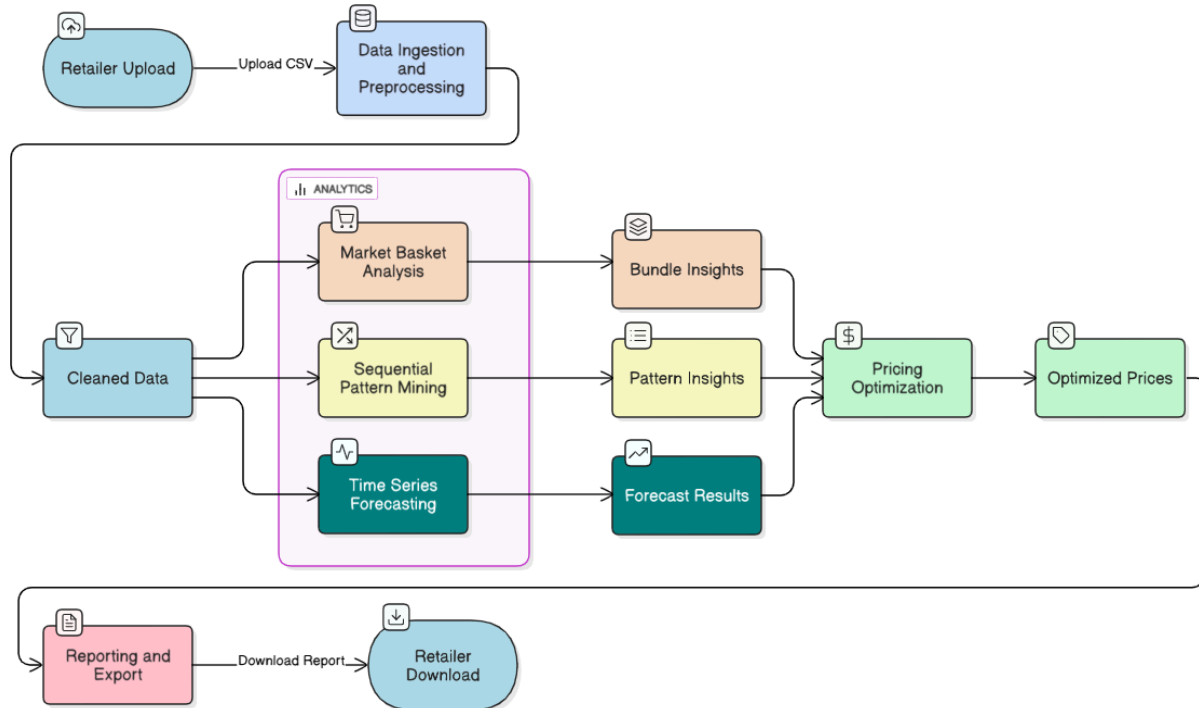


Explanation:

- The retailer uploads the CSV data file to the system.
- The system processes the data and sends outputs to the dashboard.
- Retailers view or export the analytical results.

Level 1 DFD:

This diagram breaks the system into key modules and shows how data flows internally.



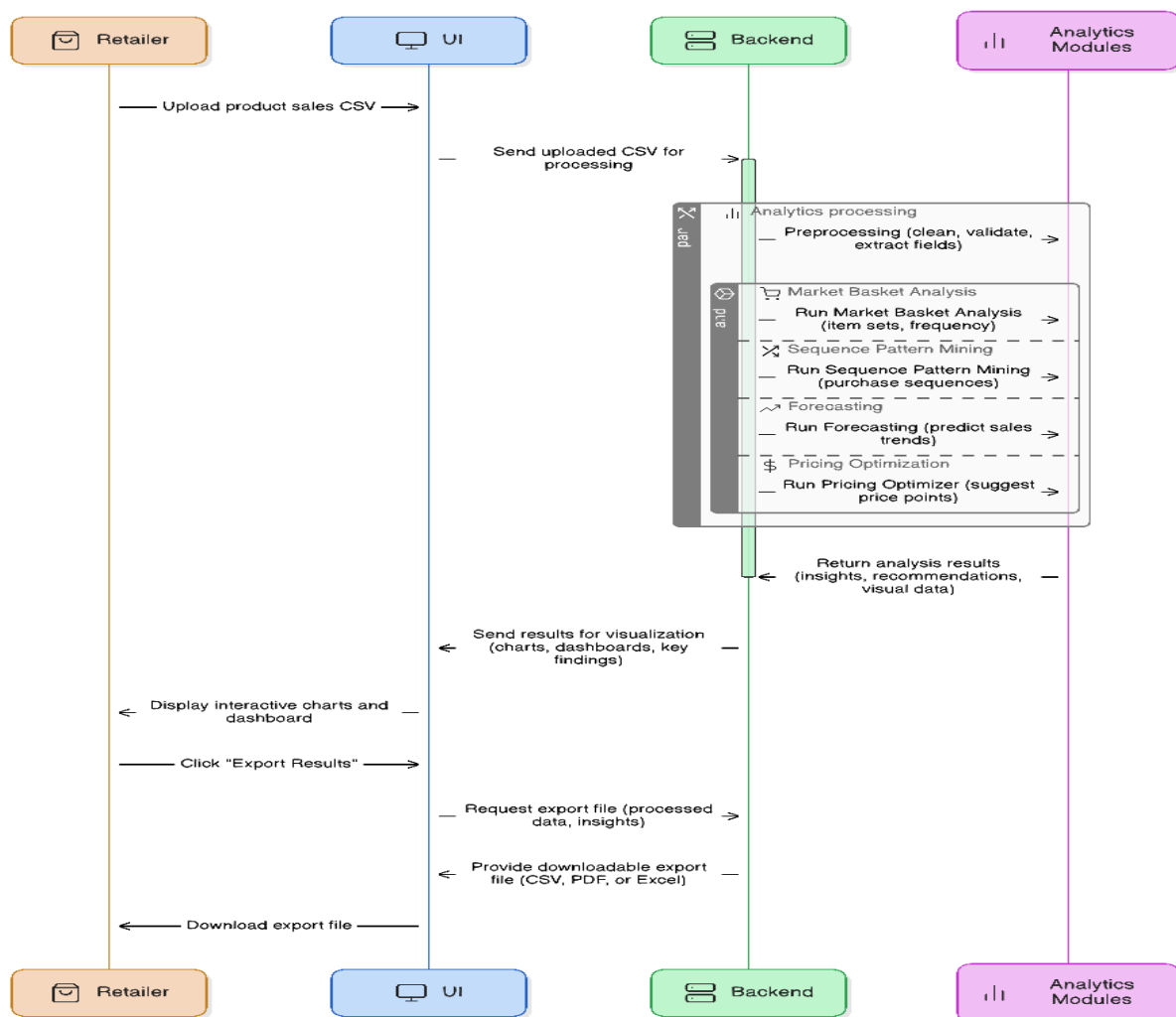
Explanation:

- **The retailer uploads** the sales data CSV file through the web interface.
- The system performs **data ingestion and preprocessing** to clean and structure the input.
- The cleaned data is sent to two analytical engines:
 - **Market Basket Analysis (MBA)** to find frequently co-purchased items.
 - **Sequential Pattern Mining (SPM)** to discover time-based purchase patterns.
- Cleaned data is also sent to the **Forecasting module**, which predicts future sales and profit trends.

- Insights from MBA/SPM and Forecasting are passed to the **Pricing Optimization module** to suggest the best prices for products or bundles.
- All outputs — **bundle insights, forecasts, and optimized prices** — are compiled into a final report by the **Reporting & Export module**.
- The **retailer views or downloads** the results through the dashboard.

6.2 Sequence Diagram

This diagram outlines the step-by-step interaction between the user and system components during a typical session.

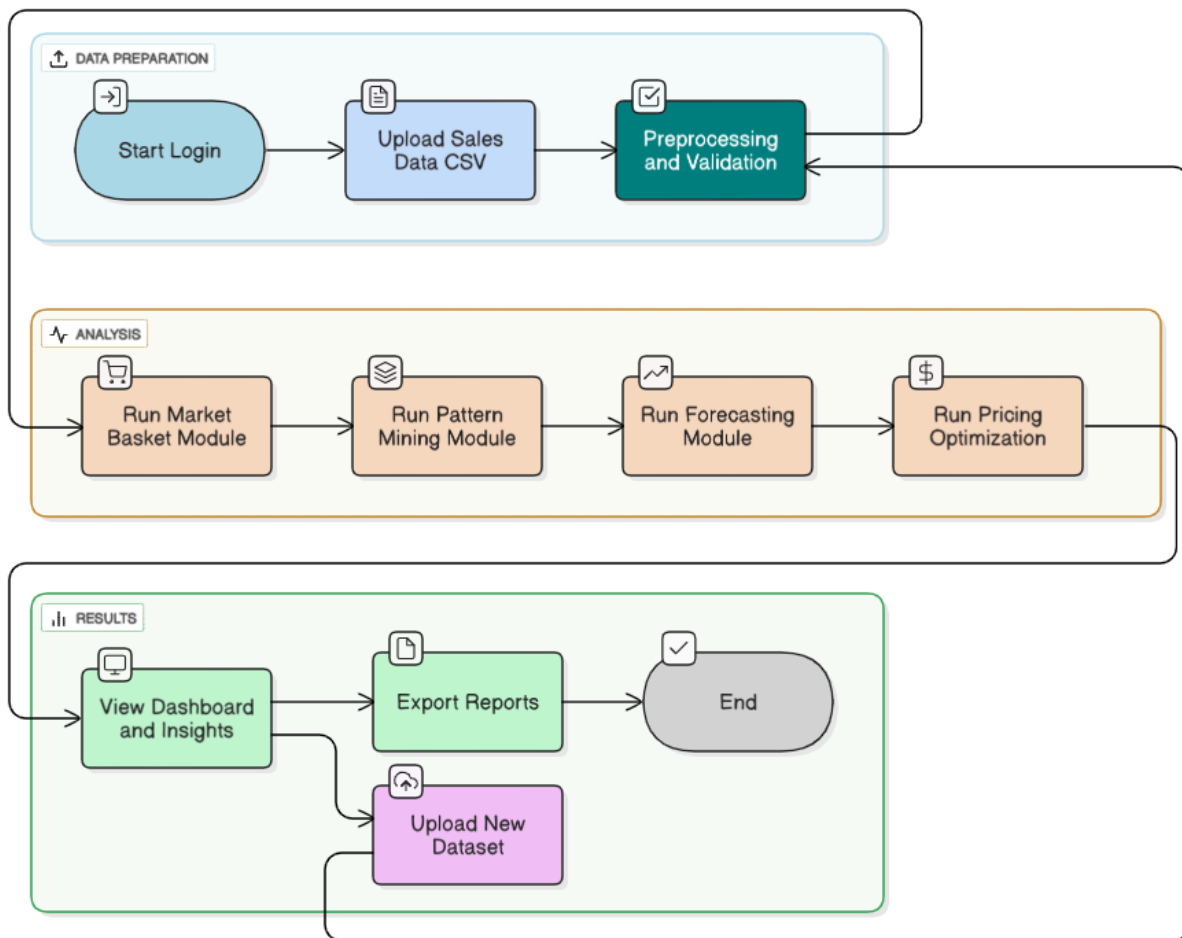


Explanation:

- Retailer uploads a file → UI forwards it to the backend.
- Backend invokes preprocessing and analytical modules.
- Results are returned to UI for visualization and download.

6.3 Activity Diagram (User Perspective)

This describes the typical flow from the retailer's point of view when using BundleWise.



7. UI and Interaction Design

7.1 UI Wireframes or Mockups

Objective:

To provide users with a clear and intuitive interface that supports smooth navigation, insightful visualizations, and easy interaction with the system's core functionalities.

Key Screens:

1. Home Page / Landing Page

- Welcome text introducing BundleWise.
- Button to "Upload Data".
- Brief description of the system's purpose.
- Links to support/help documentation.

2. Data Upload Page

- File upload component (CSV only).
- Display file format guidelines.
- Error messages for malformed or missing fields.
- Button: "Submit and Analyze".

3. Dashboard – Main Analysis Page

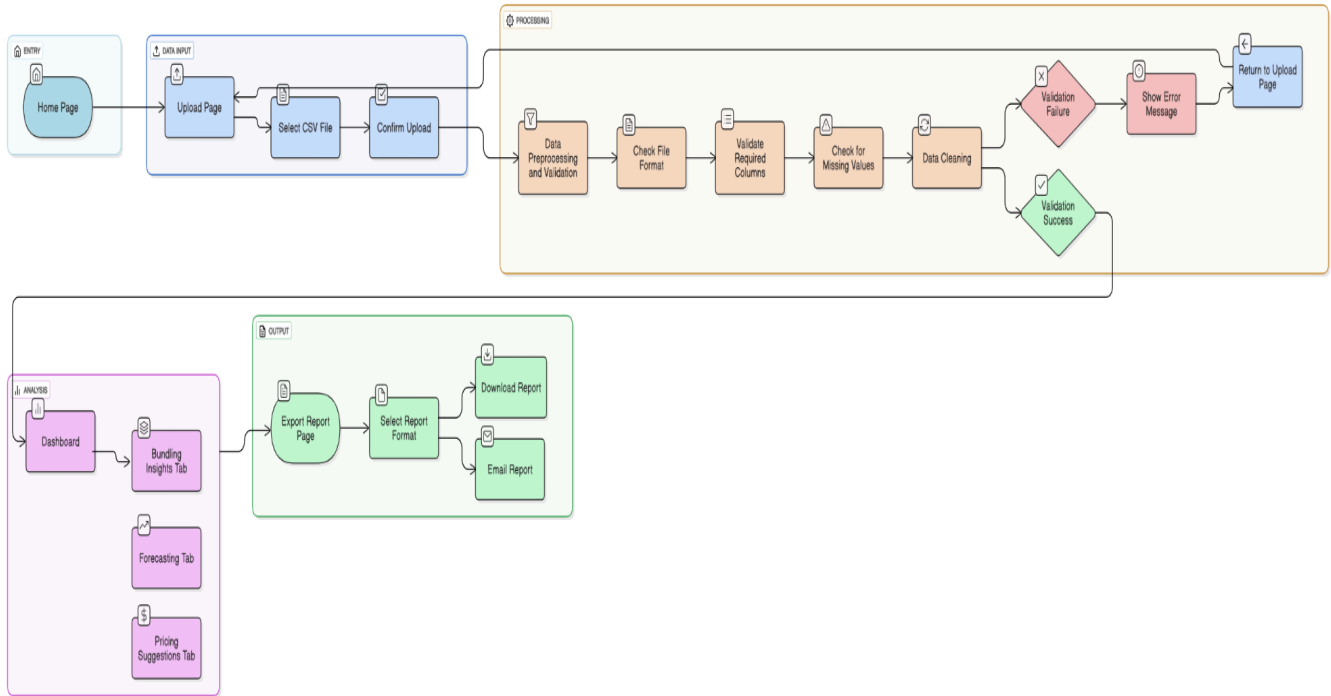
- Tabs or sections for:
 - **Bundle Insights:** Bar charts and tables showing the top frequent bundles.
 - **Sales Forecast:** Line graphs of predicted sales/profit trends.
 - **Pricing Suggestions:** Table with suggested price changes and rationales.
- Export button (PDF/Excel).
- Filters for product category, time range, and customer segment.

4. Report Export Modal

- Dropdown for file type (PDF or Excel).
- Button: "Generate Report".

7.2 Navigation Flow Diagram

This diagram shows how a user moves through the BundleWise system:



7.3 Interaction Design Principles

To ensure the BundleWise interface is accessible and practical for non-technical retail users, the following design principles are applied:

Ease of Use

- Minimal learning curve.
- Clear labeling and consistent layout.
- Step-by-step guided flow (upload → process → visualize → export).

Feedback and Validation

- Real-time feedback for uploads (e.g., “CSV uploaded successfully”).
- Error messages for incorrect formats or missing fields.

Tooltips and Help Popups

- On-hover tooltips to explain graphs, columns, and suggested actions.
- The help icon links to the FAQ and usage guide.

Responsive Design

- Fully functional across desktop, tablet, and mobile browsers.
- UI components adapt to screen sizes using frameworks like Bootstrap or Streamlit's responsive layout.

Accessibility

- High contrast for readability.
- Use of accessible fonts and button sizes.
- Keyboard navigability and screen-reader compatibility (future enhancement).

Performance-Oriented

- Loading indicators while processing data.
- The default tab displays bundling results for faster insights.

8. Security and Performance

8.1 Data Privacy and Handling

Ensuring user data's privacy and secure handling is a key concern for BundleWise. Since retailers will upload sales and transaction data, some of which may be sensitive or business-critical, the following strategies are employed:

- **Temporary File Storage:**
 - Uploaded CSV files are stored temporarily in a secure memory or local session directory.
 - Files are deleted automatically after analysis or the session ends to ensure no persistent data remains.
- **Encryption in Transit:**
 - Data uploaded via the web interface is transmitted over secure HTTPS connections.
 - This ensures protection against man-in-the-middle attacks during upload/download.
- **No Persistent User Data:**
 - BundleWise does not store user credentials or maintain persistent records of historical uploads.
 - All data is handled in-session only unless explicit logging is required (e.g., for premium features in future versions).
- **Access Control:**
 - In multi-user or cloud deployments, access to data is scoped to individual sessions, preventing cross-user access.
- **Data Validation and Sanitization:**
 - Before processing, the uploaded data is validated and sanitized to prevent injection attacks or system misuse.
 - Strict file-type checks are enforced to only allow .csv files.
- **Compliance Readiness:**
 - Though not processing personally identifiable information (PII) by default, the system can be made compliant with privacy standards like GDPR if integrated with real customer data in the future.

8.2 Performance Goals

Even with medium-to-large datasets, the system is optimized to ensure fast and responsive behavior. Key performance targets include:

- **Upload and Validation:**
 - CSV file upload and validation should complete within 15-20 seconds.
- **Preprocessing and Cleaning:**
 - Data cleaning and formatting should not exceed 10-15 seconds for standard files.
- **Sequential Pattern Mining:**
 - For 10,000–50,000 transactions, SPM algorithms (PrefixSpan or SPADE) should complete within 10 seconds.
- **Forecasting and Pricing Computation:**
 - Time-series forecasting and optimization should finish within **10 seconds**, depending on the complexity.
- **Dashboard Rendering:**
 - Initial dashboard load after analysis should occur in **<20 seconds**, with charts loading incrementally.

These performance metrics assume a cloud server with at least 4 GB RAM and 2-core CPU. Local deployments may vary slightly.

8.3 Scalability Approach

As BundleWise targets small and mid-sized retailers, it is designed to scale efficiently in terms of both data size and concurrent users:

- **Modular Backend Architecture:**
 - Modules for preprocessing, mining, forecasting, and optimization are loosely coupled, allowing parallel execution and independent scaling.
- **Cloud Deployment Compatibility:**
 - Designed for platforms like **Streamlit Cloud**, **Render**, and **Heroku**, which allow horizontal and vertical scaling based on traffic.
- **Session-Based Isolation:**
 - Each user session handles its memory space and temporary storage, making isolating and managing multiple users easier.

- **Resource Throttling (Future Enhancement):**
 - API rate limits and resource quotas will be enforced for enterprise use cases to ensure fairness and reliability.
- **Batch Processing Mode:**
 - A batch mode could be triggered for large datasets (e.g., >100k records) where processing happens asynchronously with email notifications or downloadable results.
- **Stateless Deployment:**
 - The system is designed to be stateless, allowing easy deployment behind load balancers or in containerized environments (Docker/Kubernetes for future phases).

9. Error Handling and Logging Design

Effective error handling and systematic logging are essential to ensure reliability, maintainability, and a smooth user experience. This section outlines how BundleWise handles unexpected events and system-level failures.

9.1 Handling Missing or Malformed Files

BundleWise validates all uploaded files before processing to prevent system crashes or misleading outputs due to insufficient data.

- **File Type Validation:**
 - The system only accepts .csv files.
 - Uploads of unsupported file types (e.g., .xls, .txt, .png) are rejected with a clear error message.
- **Schema Validation:**
 - Uploaded CSV files must contain mandatory columns such as:
 - Transaction ID
 - Product ID
 - Customer ID
 - Date
 - Quantity
 - Price
 - Missing or misspelled column names will trigger a validation failure message.
- **Data Format Checks:**
 - Dates are checked for correct format (e.g., YYYY-MM-DD).
 - Numerical columns (like price or quantity) are validated for non-null and positive values.
 - Null values are either flagged for imputation or removed based on severity.
- **User Feedback:**
 - If errors are detected, the system displays a detailed error report outlining:
 - Missing columns
 - Inconsistent formatting
 - Rows skipped due to invalid data

9.2 Handling Algorithm Failures

In cases where the system's analytical modules encounter failures, fallback mechanisms are used to maintain continuity and reliability.

- **SPM Failures (Pattern Mining):**
 - If the PrefixSpan or SPADE algorithm fails due to insufficient data or zero support, the system will:
 - Notify the user that no frequent patterns were found
 - Provide suggestions such as increasing the dataset size or lowering the support threshold
- **Forecasting Failures:**
 - If LSTM fails to converge (e.g., due to too few data points), the system will:
 - Automatically fall back to simpler models like ARIMA or Prophet
 - Display a warning indicating the fallback occurred
- **Optimization Failures:**
 - If the pricing model encounters an infeasible solution or fails to converge, the system will:
 - Re-run with relaxed constraints
 - Notify the user about the limitations of the output

9.3 User Notifications and Fallbacks

User awareness is prioritized during failures or warnings to avoid confusion and increase trust.

- **UI Alerts:**
 - Toast messages or modal popups notify users of any data or algorithm issues.
 - Color-coded indicators (e.g., red for errors, yellow for warnings) help highlight severity.
- **Inline Hints:**
 - Field-level tooltips or warnings appear next to the upload form when validation fails.
- **Summary Reports:**
 - At the end of each analysis session, a brief report is shown on the dashboard summarizing:
 - Number of records processed

- Any issues encountered
- Actions taken (e.g., model fallback, data skipped)

- **Recovery Tips:**

- The UI may offer suggestions like:
 - "Try uploading a file with more transaction data."
 - "Check date format and required columns."

10. Testing Overview

Testing ensures that all components of BundleWise perform as expected and provide accurate, usable, and reliable results. This section outlines the testing methodology, tools used, and key scenarios that will be validated.

10.1 Types of Testing

The following testing types will be conducted to ensure the quality and robustness of BundleWise:

1. Unit Testing

- Objective: Test individual functions/modules in isolation.
- Target: Data preprocessing functions, pattern mining, forecasting functions, pricing logic.
Example: Check if the date parsing function correctly converts multiple formats into a standard format.

2. Integration Testing

- Objective: Ensure that different modules interact correctly when combined.
- Target: Data flow from ingestion to preprocessing → pattern mining → forecasting → pricing → dashboard.
Example: Test if the pricing optimizer correctly picks up output from forecasting.

3. System Testing

- Objective: Verify that the entire system works as a whole.
- Target: Full end-to-end test from data upload to report generation.
- Example: Upload CSV, run analysis, view dashboard, and export report.

4. Usability Testing

- Objective: Test the user interface for ease of use, clarity, and user experience.
- Target: Non-technical users (retailers) navigating the dashboard and interacting with outputs.
- Example: Check if users can upload files easily and interpret forecast graphs without technical assistance.

5. Performance Testing

- Objective: Assess how the system performs under load or with large datasets.
- Target: Response time for 10k–100k records, dashboard rendering speed.

- Example: Measure the time the LSTM model takes to generate forecast on 1-year data.

10.2 Testing Tools

The following tools and platforms will be used during the testing phases:

Tool	Purpose
PyTest	Unit and integration testing of Python code
Selenium	Automated UI testing (e.g., file upload, button clicks)
Google Colab	Manual testing and notebook-based validation
Streamlit Test	UI-level mock testing (Streamlit scripts)
CSV Validator	Pre-check CSV formats and simulate bad inputs
Browser DevTools	Performance profiling and debugging

10.3 Sample Test Scenarios

Test Scenario	Expected Outcome
Uploading a well-structured CSV file	The file is accepted, cleaned, and processed successfully
Uploading a malformed CSV (missing headers)	The system shows an error message with an explanation

Upload with missing values	System handles missing data gracefully (impute or notify)
Running Sequential Pattern Mining	The system returns frequently occurring product sequences
Running Time Series Forecasting (Prophet)	Displays a forecast line chart without error
LSTM model fails to converge	System falls back to Prophet with notification
Price optimization with invalid constraints	System reports inability to find optimal pricing, suggests review
Exporting results in PDF or Excel	The user successfully downloads a formatted report
UI Load Test with 100k records	Dashboard loads within an acceptable time (<10 seconds ideally)
Navigating between dashboard sections	All UI components respond correctly and intuitively

11. Future Work and Limitations

This section outlines the system's current limitations and identifies areas for future improvement and enhancement. These insights will guide future development cycles as user needs evolve and new technological opportunities arise.

11.1 Known Limitations

Despite its robust analytical capabilities, the current version of BundleWise has several limitations due to time, resource, or technical constraints:

1. No Real-Time POS Integration

- Currently, BundleWise operates using manual CSV uploads.
- It does not yet support direct data ingestion from live Point-of-Sale (POS) systems or ERPs, which limits real-time analysis.

2. Static Seasonal/Event Data

- Forecasting models rely on static or manually input event data.
- There is no automatic incorporation of calendars, festivals, or local market events that could impact trends.

3. No User Authentication

- There is no built-in login system or multi-user session management.
- All sessions are temporary, and shared access may be a concern in collaborative environments.

4. Limited Model Interpretability

- While results are visually presented, some advanced models (e.g., LSTM) may not offer explainability unless SHAP or LIME is integrated later.

5. Cloud Hosting Limits

- Free-tier deployments (e.g., on Streamlit Cloud) may restrict execution speed, memory usage, or session duration, especially when handling large datasets.

6. No Database Integration

- The system does not store uploaded data or analysis results in a persistent database.
- All processing is session-based and temporary.

11.2 Future Enhancements

Based on current limitations and user feedback, the following enhancements are planned or proposed for future releases:

1. Real-Time Data Integration

- Enable API-based integration with POS, ERP, or e-commerce platforms like Shopify, WooCommerce, or Square for live data ingestion.

2. User Authentication & Profiles

- Implement login, registration, and role-based access.
- Allow users to save historical analyses, upload multiple datasets, and resume sessions.

3. Database Support

- Add persistent data storage using PostgreSQL, Firebase, or MongoDB.
- Enable user-specific dashboards and long-term data tracking.

4. Advanced Event Forecasting

- Integrate Google Calendar, national holiday APIs, or event feeds to improve seasonality handling in time series forecasting.

5. Mobile Optimization

- Improve responsiveness and layout for mobile users.
- Offer a lightweight version of the dashboard for tablets or phones.

6. AI Explainability

- Integrate SHAP (SHapley Additive exPlanations) or LIME to interpret model decisions and recommendations.

7. Support for Unstructured Inputs

- In future modules, use basic NLP models to interpret semi-structured files or customer feedback data.

8. Custom Bundle Builder

- Allow users to manually create product bundles and simulate price adjustments based on predicted performance.